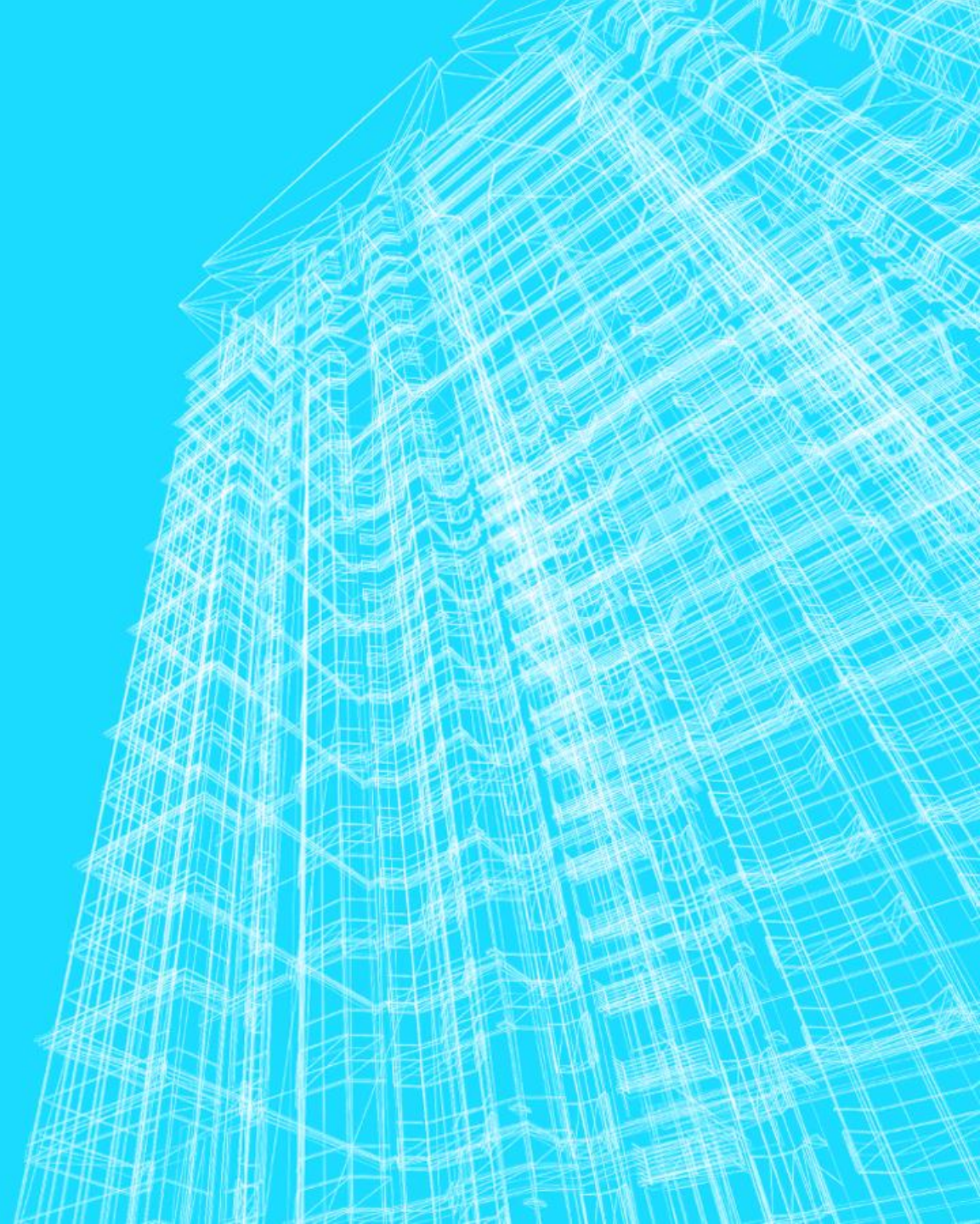
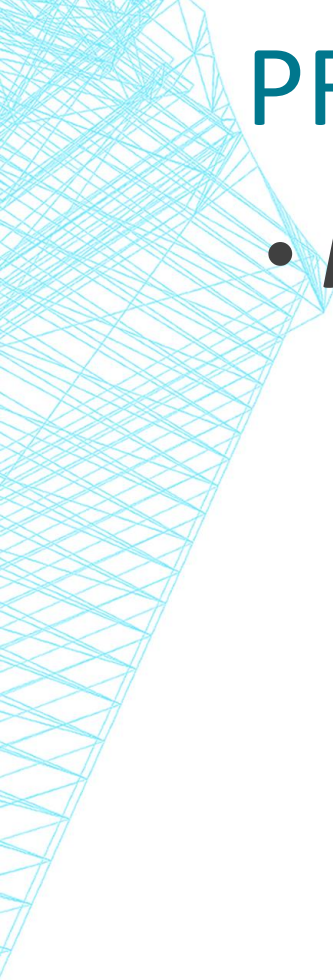


PROGRAMAÇÃO DINÂMICA

João Pedro Oliveira Batisteli





PROGRAMAÇÃO DINÂMICA?!?!

- *Programação?!?!*

PROGRAMAÇÃO DINÂMICA?!?!

- ***Programação?!?!***

- substantivo feminino.
- 1. Ato ou efeito de programar; plano; programa; conjunto de programas ou planos.

PROGRAMAÇÃO DINÂMICA?!?!

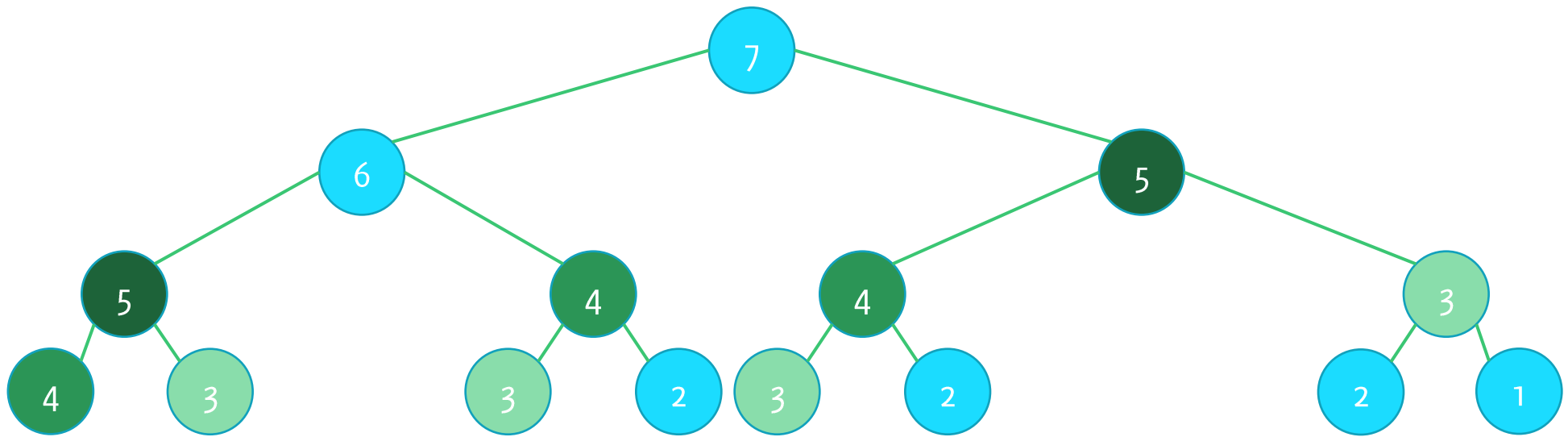
- ***Programar?!?!***

- verbo transitivo.
- 2. Estabelecer um plano para fazer algo.
- 5. Submeter ou submeter-se a uma rotina ou aprendizagem para reagir de determinada maneira.

PROGRAMAÇÃO DINÂMICA

- Técnica de projeto de algoritmos relacionada com um método de solução baseado em uma tabela.
 - Memória dos resultados;
- Tipicamente utilizada na resolução de problemas combinatórios.
- Útil quando há sobreposição de subproblemas na resolução recursiva.

```
int Fib(int n){
    if (n <= 2) return 1;
    return Fib(n - 1) + Fib(n - 2);
}
```



EXEMPLO: FIBONACCI

```
int Fib(int n){  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```

- Sobreposição de subproblemas;
- Não há memória dos resultados;
- Método de complexidade exponencial.

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```


EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

0	1	2	3	4	5	6
0	0	0	0	0	0	0

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];
```

```
    results[1] = 1; results[2] = 1;
```

```
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }
```

```
    return results[n];
```

```
}
```

N=6

0	1	2	3	4	5	6
0	1	1	0	0	0	0

ESEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=3

0	1	2	3	4	5	6
0	1	1	0	0	0	0

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=3

0	1	2	3	4	5	6
0	1	1	2	0	0	0

ESEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=4

0	1	2	3	4	5	6
0	1	1	2	0	0	0

ESEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=4

0	1	2	3	4	5	6
0	1	1	2	3	0	0

ESEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=5

0	1	2	3	4	5	6
0	1	1	2	3	0	0

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

N=6

i=5

0	1	2	3	4	5	6
0	1	1	2	3	5	0

EXEMPLO: FIBONACCI

```
int fib_PD(int n){  
    int[] results = new int[n + 1];  
  
    results[1] = 1; results[2] = 1;  
  
    for (int i = 3; i <= n; i++) {  
        results[i] = results[i-1] + results[i-2];  
    }  
    return results[n];  
}
```

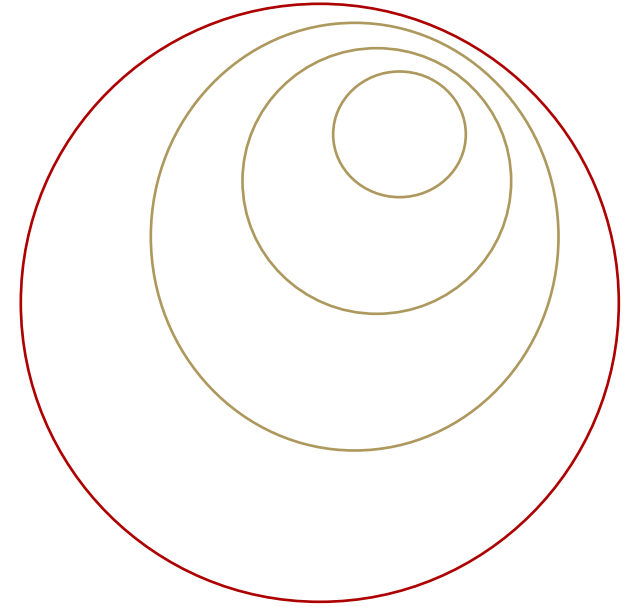
N=6

i=6

0	1	2	3	4	5	6
0	1	1	2	3	5	0

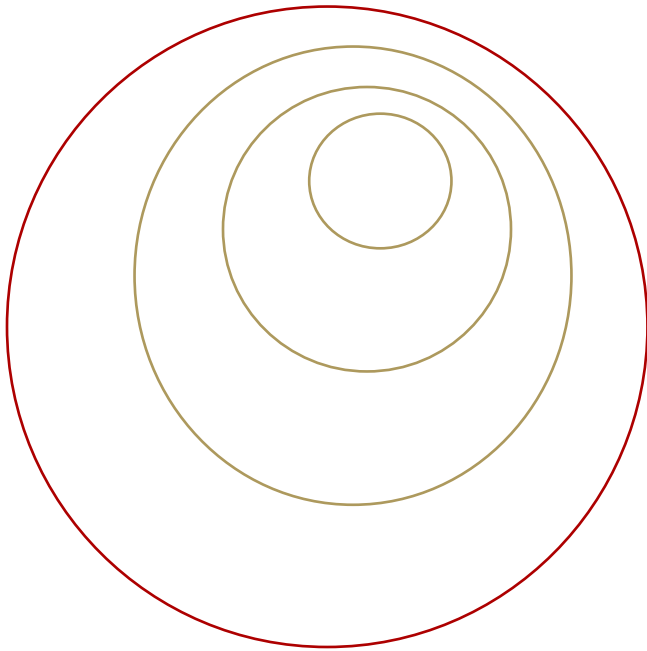
PROGRAMAÇÃO DINÂMICA

- A programação dinâmica divide o problema em problemas sobrepostos:
 - Subestrutura ótima;
 - Abordagem *bottom-up*.

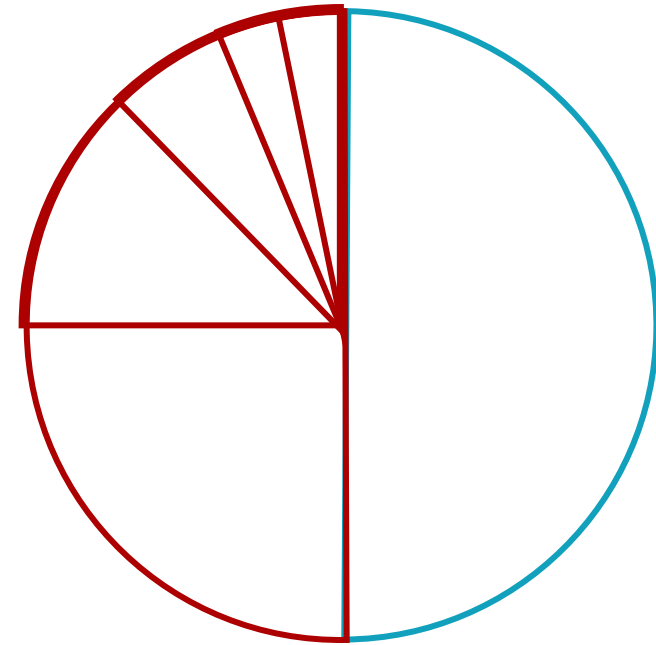


PD X D&C

- Programação dinâmica:
bottom-up.



- Divisão e conquista:
top-down.



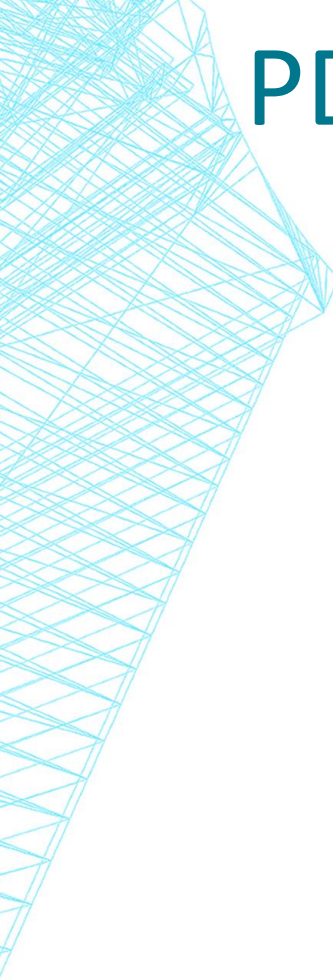
CUIDADOS NA APLICAÇÃO DA PD

- **Tamanho do espaço de instâncias (n) reduzido**
 - O custo de inicialização (*overhead*) e alocação de estruturas pode superar o tempo de processamento bruto.
- **Análise de Constantes Ocultas na Complexidade**
 - O tempo de execução real da Programação Dinâmica (ex. $O(n)$) pode ser superior ao da Recursão Pura (ex. $O(2^n)$) para entradas pequenas devido a fatores constantes.
- **Compromisso Tempo vs. Espaço (Trade-off)**
 - A redução drástica da complexidade de tempo geralmente exige um consumo extra de memória para o armazenamento de subproblemas (tabelas de memoização ou tabulação).

PROGRAMAÇÃO DINÂMICA

- Tabela: memória de cálculo.
 - Linhas: instâncias.
 - Colunas: valores possíveis.
 - Função: compara ou agrega a solução anterior.

PD – ALGORITMO GERAL



Criar uma tabela de instâncias x valores;
Inicializar primeira linha e primeira coluna;
Para cada linha,
 Para cada coluna,
 Comparar o resultado anterior com o
 resultado ao incluir a instância atual;
 Armazenar o melhor resultado;
O resultado estará no final da tabela;
Reconstruir o caminho do resultado (ou consultar
objeto);